

Trabalho Prático 2 de Grafos - Métodos de segmentação de imagens

Augusto Stambassi Duarte  [Pontifícia Universidade Católica de Minas Gerais (PUC Minas) | asduarte@sga.pucminas.br]
Davi Cândido de Almeida  [Pontifícia Universidade Católica de Minas Gerais (PUC Minas) | davi.almeida@sga.pucminas.br]
Gabriela de Assis dos Reis  [Pontifícia Universidade Católica de Minas Gerais (PUC Minas) | gabriela.reis@sga.pucminas.br]
João Pedro Torres  [Pontifícia Universidade Católica de Minas Gerais (PUC Minas) | joao.torres.1060863@sga.pucminas.br]
Lucas Carneiro Nassau Malta  [Pontifícia Universidade Católica de Minas Gerais (PUC Minas) | lcnmalta@sga.pucminas.br]
Vitor Leite Setragini  [Pontifícia Universidade Católica de Minas Gerais (PUC Minas) | 1526536@sga.pucminas.br]

✉ Instituto de Ciências Exatas e Informática, Pontifícia Universidade Católica de Minas Gerais (PUC Minas), Av. Dom José Gaspar, 500, Coração Eucarístico, Belo Horizonte, MG 30535-901, Brasil.

Resumo. Este artigo apresenta o desenvolvimento e a avaliação experimental de métodos de segmentação de imagens baseados em grafos, utilizando tanto árvores geradoras mínimas (MST, do inglês *Minimum Spanning Tree*) quanto arborescências geradoras mínimas (MSA, do inglês *Minimum Spanning Arborescence*). A partir de imagens na forma PPM, modelamos cada pixel como um vértice de um grafo ponderado, em que as arestas conectam pixels adjacentes e os pesos refletem a dissimilaridade entre eles. Com base nessa estrutura, implementamos um segmentador com o método de Felzenszwalb e Huttenlocher para MST e a versão de Tarjan do algoritmo de Edmonds. Comparamos o desempenho e o comportamento desses algoritmos em grafos derivados de imagens reais, analisando custo computacional, estabilidade e qualidade da segmentação. Os resultados indicam que a MST é competitiva em simplicidade e velocidade, enquanto as MSAs permitem estruturar a segmentação em torno de uma raiz e explorar propriedades direcionadas do grafo.

Keywords: Segmentação de Imagens, Árvores Geradoras Mínimas, Arborescências Geradoras Mínimas, Algoritmos em Grafos, C++.

1 Introdução

A segmentação de imagens é uma etapa fundamental em visão computacional, pois permite decompor uma imagem em regiões coerentes, facilitando tarefas como detecção de objetos, análise estrutural e extração de características. Entre as diversas abordagens existentes, os métodos baseados em grafos se destacam pela capacidade de representar explicitamente relações locais entre pixels: cada pixel é modelado como um vértice e as arestas conectam pixels vizinhos, com pesos que refletem diferenças de cor ou intensidade [Cousty *et al.*, 2018].

Nesse contexto, o método de Felzenszwalb e Huttenlocher tornou-se uma referência ao utilizar árvores geradoras mínimas (MST, do inglês *Minimum Spanning Tree*) para guiar a fusão de componentes em um grafo de pixels. A partir de uma MST construída sobre a imagem, o algoritmo decide quando unir ou separar regiões com base em um critério que compara o peso das arestas com a variação interna dos componentes, resultando em segmentações eficientes, com boa preservação de bordas e custo praticamente linear.

O problema análogo em grafos direcionados é o da arborescência geradora mínima (MSA, do inglês *Minimum Spanning Arborescence*). Dado um grafo direcionado ponderado e uma raiz r , busca-se uma árvore direcionada de menor custo que alcance todos os vértices. Esse problema foi formulado e resolvido por Edmonds [Edmonds, 1967], em um algoritmo conhecido como Chu–Liu/Edmonds, e posteriormente refinado por Tarjan [Tarjan, 1977] e por Gabow *et al.* [Gabow *et al.*, 1986]. Trabalhos recentes, como o de

Böther *et al.* [Böther *et al.*, 2023], revisitam essas abordagens e discutem variantes modernas como a GGST, reforçando o interesse em estruturas direcionadas de custo mínimo.

Apesar desse corpo teórico, o uso de MSAs em segmentação de imagens ainda é pouco explorado. Intuitivamente, a estrutura enraizada de uma MSA introduz assimetria na conectividade, potencialmente produzindo segmentações diferentes daquelas derivadas de MSTs não direcionadas. Neste trabalho, investigamos essa diferença implementando um segmentador baseado no método de Felzenszwalb e Huttenlocher para MST e uma implementação funcional do algoritmo de Edmonds para MSA, ambos sobre a mesma modelagem de imagem como grafo ponderado. Nosso objetivo é comparar qualitativamente as segmentações geradas e discutir o comportamento computacional das duas estratégias.

2 Fundamentação Teórica

2.1 Segmentação por Grafos e MST

Na abordagem por grafos, uma imagem é representada por um grafo ponderado $G = (V, E)$, em que cada pixel é um vértice $v \in V$ e as arestas conectam pixels adjacentes, tipicamente em vizinhança 4 ou 8. O peso de uma aresta (u, v) codifica a dissimilaridade entre os pixels, por exemplo, a distância euclidiana entre vetores de cor. A segmentação consiste em particionar V em componentes que sejam internamente homogêneas e separadas por fronteiras com grande variação

[Cousty *et al.*, 2018].

A árvore geradora mínima é uma estrutura clássica para explorar essa ideia. Dado um grafo não direcionado e ponderado, uma MST conecta todos os vértices com o menor custo total possível. Felzenszwalb e Huttenlocher utilizam a MST do grafo de pixels como base de um algoritmo de segmentação: as arestas da MST são processadas em ordem crescente de peso e, para cada aresta que conecta dois componentes C_1 e C_2 , o algoritmo decide fundi-los ou não comparando o peso da aresta com uma medida de variação interna de cada componente acrescida de um termo de escala. A decisão é implementada sobre uma estrutura de união e busca (*Union-Find*), o que garante eficiência prática próxima de $O(E \log E)$ (dominada pela ordenação das arestas) e, mais precisamente, $O(|E|\alpha(|V|))$ para a fase de união e busca.

2.2 Arborescências Geradoras Mínimas

No caso direcionado, o problema correspondente é o da arborescência geradora mínima. Seja $G = (V, E)$ um grafo direcionado ponderado e $r \in V$ uma raiz. Uma arborescência enraizada em r é um subgrafo em que r tem grau de entrada zero, cada vértice $v \neq r$ tem exatamente uma aresta de entrada e existe um caminho dirigido de r até qualquer outro vértice. A MSA é a arborescência que minimiza o somatório dos pesos das arestas.

O algoritmo de Edmonds [Edmonds, 1967] resolve esse problema selecionando, para cada vértice diferente da raiz, a aresta de menor peso que entra nele. Se o conjunto dessas arestas não contém ciclos, obtemos diretamente a arborescência mínima. Caso haja ciclos, cada ciclo é contraído em um supervértice e o processo é repetido recursivamente em um grafo reduzido. Ao final, os ciclos são expandidos para reconstruir a solução no grafo original. A complexidade do método é $O(nm)$, em que $n = |V|$ e $m = |E|$.

Tarjan [Tarjan, 1977] e Gabow *et al.* [Gabow *et al.*, 1986] propuseram variantes que reorganizam a detecção e a contração de ciclos, além de utilizar estruturas de dados mais elaboradas, obtendo melhorias tanto teóricas quanto práticas. Böther *et al.* [Böther *et al.*, 2023] revisitam essas contribuições, com ênfase em implementações modernas e na variante GGST. Embora essas abordagens sejam amplamente discutidas na literatura de algoritmos, sua aplicação em segmentação de imagens ainda é rara, o que motiva a comparação exploratória realizada neste trabalho entre MSTs e MSAs construídas sobre o mesmo grafo de pixels.

3 Modelagem e Implementação

Nesta seção, descrevemos o processo de extensão da biblioteca de grafos utilizada para suportar a segmentação de imagens, detalhando a modelagem da imagem como grafo e a implementação dos segmentadores baseados em MST e MSA.

3.1 Infraestrutura e Interface de Segmentação

O trabalho reutiliza uma infraestrutura de grafos em C++ baseada na classe `GrafoLista` para representar grafos por meio de listas de adjacências ponderadas. Cada vértice corresponde a um pixel, o que facilita o mapeamento direto. Sobre essa estrutura, foi definida a interface `ISegmentador`, que unifica a funcionalidade e expõe o método de processamento:

```
ResultadoSegmentacao
segmentar(GrafoLista* grafo, double k,
int minSize);
```

O tipo `ResultadoSegmentacao` associa cada vértice a um identificador de componente. A partir dessa interface, foram implementados o `SegmentadorMST` e o `SegmentadorMSA`, permitindo a comparação dos algoritmos sobre o mesmo modelo de grafo.

3.2 Modelagem da Imagem como Grafo

A conversão da imagem (formato PPM) em grafo é gerenciada pelo módulo `InterfaceImagem`. O mapeamento de pixels para vértices é dado por:

$$u = y \cdot \text{largura} + x,$$

de modo que o grafo possui $|V| = \text{largura} \times \text{altura}$ vértices. A cor do pixel é codificada em um único valor em ponto flutuante.

As arestas são definidas a partir da **vizinhança 4-conexa**. O peso da aresta $w(u, v)$ é dado pela **distância euclidiana no espaço RGB** entre os vetores de cor de origem u e destino v :

$$w(u, v) = \sqrt{(R_u - R_v)^2 + (G_u - G_v)^2 + (B_u - B_v)^2}.$$

O grafo é configurado como não direcionado para a MST e com arestas em ambas as direções para a MSA.

3.3 Segmentação Baseada em MST (Critério de Fusão)

A classe `SegmentadorMST` implementa o método de Felzenszwalb e Huttenlocher. As arestas do grafo são extraídas e ordenadas por peso, e a estrutura de *Union-Find* é utilizada para gerenciar os componentes.

O critério de fusão compara o peso w da aresta com um limiar $\text{MInt}(C_1, C_2)$. Seja $\text{Int}(C)$ o máximo peso de aresta interna em C e a função de escala $\tau(C) = k/|C|$. Os componentes C_1 e C_2 são unidos apenas se $w \leq \text{MInt}(C_1, C_2)$, onde:

$$\text{MInt}(C_1, C_2) = \min(\text{Int}(C_1) + \tau(C_1), \text{Int}(C_2) + \tau(C_2)).$$

Ao final, os componentes são rotulados (via busca em largura) e convertidos em `ResultadoSegmentacao`.

3.4 Segmentação Baseada em MSA

A segmentação baseada em MSA é realizada sobre o grafo direcionado. O módulo `SegmentadorMSA` acopla uma implementação funcional do **algoritmo de Edmonds**, responsável por selecionar as arestas de menor peso de entrada, detectar e contrair ciclos (com o auxílio de uma raiz virtual) e, por fim, recuperar a arborescência mínima. O resultado é processado para `ResultadoSegmentacao`, permitindo a comparação qualitativa das segmentações obtidas por MST e MSA.

4 Experimentos e Resultados

Nesta seção, avaliamos o desempenho dos algoritmos de segmentação MST e MSA utilizando uma imagem de cena natural complexa (resolução 673×425), que apresenta desafios de textura (pelagem dos animais) e iluminação. O objetivo foi analisar como cada método isola os objetos de interesse e define suas fronteiras.

4.1 Configuração dos Parâmetros

Os parâmetros de escala k e tamanho mínimo de componente (*minSize*) foram ajustados individualmente para cada método a fim de obter a melhor representação visual possível dos objetos:

- **MST (Felzenszwalb):** Foi necessário um valor alto de $k = 900$ e *minSize* = 900. Isso indica que, para ignorar a variação de alta frequência da textura dos pelos e agrupar o coelho inteiro em uma única região, o algoritmo precisou de um limiar de relaxamento agressivo.
- **MSA (Edmonds):** O método mostrou-se extremamente sensível, operando com $k = 10$ e *minSize* = 600. Valores de k superiores a isso tendiam a fundir a imagem inteira em um único componente, sugerindo que a estrutura direcionada propaga conexões de forma mais rígida.

4.2 Análise Visual: MST

A segmentação baseada em MST (Figura 1) resultou em regiões compactas e organicamente distribuídas. O algoritmo funcionou efetivamente como um filtro de suavização, removendo os detalhes finos da pelagem e gerando um efeito de “pintura a óleo”.

Observando a imagem de bordas (Figura 1c), nota-se que os contornos principais dos coelhos e da janela foram preservados com curvas suaves e contínuas. A alta tolerância ($k = 900$) permitiu que variações locais de cor fossem absorvidas dentro dos componentes, evitando a super-segmentação dentro dos corpos dos animais.

4.3 Análise Visual: MSA

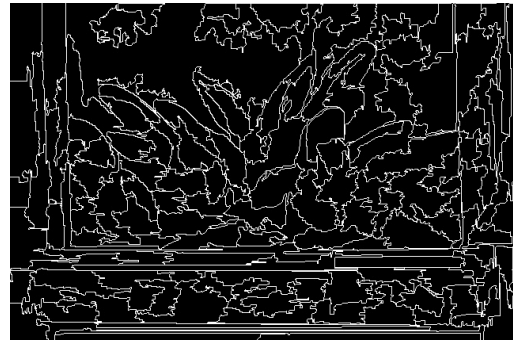
A abordagem por MSA (Figura 2) produziu um resultado visualmente distinto, caracterizado por uma fragmentação mais geométrica e menos orgânica. Apesar do parâmetro k



(a) Imagem Original



(b) Segmentação MST ($k = 900$)



(c) Bordas MST

Figure 1. Resultados da MST. O método abstraiu bem a textura, criando regiões coesas.

significativamente menor ($k = 10$), o algoritmo teve dificuldade em formar grandes componentes homogêneos nas áreas texturizadas.

A análise das bordas (Figura 2b) revela contornos mais irregulares e ruidosos em comparação à MST. Nota-se a presença de artefatos horizontais e verticais, especialmente na madeira inferior e no fundo escuro. Isso evidencia a dependência da raiz e a natureza direcionada das arestas: o crescimento das regiões é restringido pela direção de menor custo, o que pode impedir a fusão de pixels vizinhos que seriam trivialmente unidos em um grafo não-direcionado.

4.4 Discussão e Desempenho Computacional

A análise dos resultados revelou um claro trade-off entre eficiência e qualidade visual em favor da MST. Em termos de custo computacional, a MST ($O(E \log E)$) demonstrou superioridade, processando a imagem em apenas 287 milissegundos, contrastando com os 1.822 milissegundos exigidos pela MSA ($O(nm)$). Qualitativamente, a coerência da MST foi alcançada mediante um limiar de relaxamento agressivo ($K = 900$) necessário para agrupar texturas. A MSA, de-

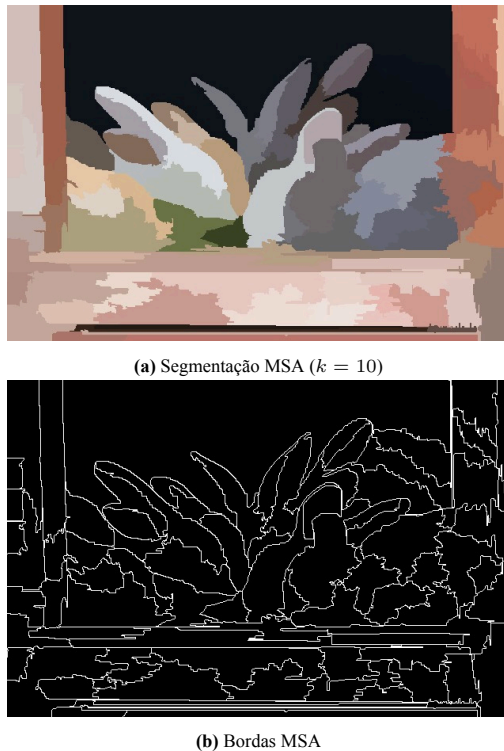


Figure 2. Resultados da MSA. Nota-se uma segmentação mais ruidosa e artefatos geométricos, reflexo da estrutura direcionada.

vido à direcionalidade, impôs uma sensibilidade de fusão muito maior, exigindo um parâmetro drasticamente baixo ($K = 10$), o que resultou em uma segmentação mais fragmentada e geométrica. Conclui-se que a MSA é menos adequada que a MST para segmentação geral de cenas orgânicas, devido tanto à sua complexidade computacional elevada quanto ao comportamento paramétrico instável.

5 Conclusão

Este trabalho investigou o impacto da estrutura gráfica na segmentação de imagens, comparando a Árvore Geradora Mínima (MST), baseada em grafos não-direcionados, com a Arborescência Geradora Mínima (MSA), baseada em grafos direcionados. Os experimentos demonstraram que a escolha da estrutura impõe diferenças críticas tanto no comportamento dos parâmetros quanto na qualidade visual da segmentação.

O achado mais significativo reside na disparidade paramétrica: enquanto a MST exigiu um alto parâmetro de escala ($K = 900$) para agrupar efetivamente objetos coesos e absorver ruídos de textura, a MSA revelou uma sensibilidade de fusão drasticamente superior, exigindo um K muito baixo ($K = 10$). Esta diferença evidencia que a direcionalidade do grafo facilita a propagação de fronteiras, tornando o algoritmo MSA inerentemente mais restritivo.

Qualitativamente, a estrutura isotrópica da MST produziu resultados superiores, gerando regiões compactas e contornos suaves, mais coerentes com a percepção humana em cenas naturais. A MSA, em contrapartida, resultou em segmentações fragmentadas e geometrizadas, limitando sua eficácia em texturas orgânicas e complexas. Conclui-se que a MST permanece o método mais robusto e eficiente para a

segmentação de propósito geral.

Como trabalho futuro, sugere-se a otimização do custo computacional da MSA e a exploração de seu potencial em domínios onde a direcionalidade possui significado físico (como em imagens médicas ou de fluxo), utilizando métricas de custo adaptativas para mitigar a sensibilidade excessiva do método.

Organização do trabalho

Todos os autores contribuíram em diferentes etapas do trabalho. Augusto Stambassi Duarte ficou responsável pela adaptação da biblioteca de grafos pré-existente e pela modelagem das imagens como grafos ponderados, bem como o processo de reversão do grafo em imagem. Lucas Carneiro Nassau Malta implementou e validou o segmentador baseado em MST, incluindo experimentos iniciais de ajuste do parâmetro de escala. Davi Cândido de Almeida e Gabriela de Assis dos Reis implementou a versão do algoritmo de Edmonds para MSA. Adaptações do modelo aos métodos de Tarjan foram também incorporados por Davi Cândido, enquanto João Pedro Torres se concentrou na fundamentação teórica e otimização das variantes. Vitor Leite Setragni e Davi Cândido coordenou a integração dos métodos na interface de visualização de imagens. Lucas Carneiro liderou a organização e análise dos experimentos. Todos os autores contribuíram para a redação, revisão e aprovação da versão final do manuscrito.

Disponibilização do material

Os códigos gerados e analisados durante o presente estudo estão disponíveis em: <https://github.com/VitorSetragni/TP2-Grafos>.

Testes adicionais e demais resultados podem ser acessados em: <https://github.com/VitorSetragni/TP2-Grafos/tree/main/Assets>

References

- Böther, M. *et al.* (2023). Efficiently computing directed minimum spanning trees. In *Proceedings of the Symposium on Algorithm Engineering and Experiments (ALENEX)*, pages 86–95.
- Cousty, J., Najman, L., Kenmochi, Y., and Guimarães, S. J. F. (2018). Hierarchicalfelzenszwalb2004efficient segmentations with graphs: Quasi-flat zones, minimum spanning trees, and saliency maps. *Journal of Mathematical Imaging and Vision*, 60(4):479–502.
- Edmonds, J. (1967). Optimum branchings. *Journal of Research of the National Bureau of Standards, Section B*, 71B(4):233–240.
- Gabow, H. N., Galil, Z., Spencer, T., and Tarjan, R. E. (1986). Efficient algorithms for finding minimum spanning trees in undirected and directed graphs. *Combinatorica*, 6(2):109–122.
- Tarjan, R. E. (1977). Finding optimum branchings. *Networks*, 7:25–35.